SE Lab 3: Architectural Pattern Selection

Name: Rahul B

SRN: PES2UG24CS393

Section: G

1. EXECUTIVE OVERVIEW & LEARNING OBJECTIVES

The objective of this laboratory exercise is to systematically evaluate competing software architectural styles (Layered, Microservices, and Client-Server), select the optimal architectural pattern for the assigned software systems, and construct formal UML Component Diagrams depicting components, provided/required interfaces (ball-and-socket notation), ports, and dependencies.

This submission delivers the complete architectural justifications, component models, and UML diagrams for:

1. **Part A (Assigned Semester Project):** Problem Statement #25 – *Municipal Solid Waste Route Optimizer*.
2. **Part B (In-Lab Handout Scenario):** *Self-Service Coffee Kiosk System* (incorporating the two assigned components and interface).

2. ASSIGNED SCENARIO OVERVIEW – MUNICIPAL SOLID WASTE ROUTE OPTIMIZER (PS #25)

Problem Context: A smart-city sanitation management platform that continuously ingests ultrasonic fill-level telemetry from 10,000 smart trash bins distributed citywide, plans optimized daily collection routes for a fleet of 50 garbage trucks, and tracks citizen complaints regarding missed garbage pickups. The primary human actors are the *Sanitation Supervisor* and the *Garbage Truck Driver*, alongside city residents.

System Requirements Summary:

- **FR-001 (Automated Threshold-Based Routing) [Priority: High]:** The system shall generate daily collection routes including only bins exceeding an 80% fill threshold (or manually flagged for emergency collection).
- **FR-002 (Real-Time Sensor Ingestion) [Priority: High]:** The system shall continuously ingest ultrasonic fill-level telemetry from 10,000 smart trash bins via MQTT/HTTPS.
- **FR-003 (Driver Manifest & GPS Telematics) [Priority: High]:** The system shall dispatch turn-by-turn route manifests to truck drivers' mobile units and track collection progress via real-time GPS coordinates.
- **FR-004 (Citizen Complaint Portal) [Priority: Medium]:** The system shall provide a web and mobile portal allowing residents to submit missed-pickup complaints with geolocation coordinates and photographic evidence.
- **FR-005 (Supervisor Fleet Dashboard) [Priority: Medium]:** The system shall provide an administrative dashboard displaying real-time bin fullness heatmaps, truck telemetry, route efficiency metrics, and manual dispatch overrides.
- **NFR-001 (Performance & Optimization Latency):** Route recalculation for a fleet of 50 trucks across 10,000 bins shall execute in under 45 seconds under simulated peak operational load.
- **NFR-002 (Security, Access Control & Reliability):** The system shall enforce Role-Based Access Control (RBAC), end-to-end TLS 1.3 encryption for sensor telemetry, and guarantee 99.9% uptime during operational collection windows.

Core Engineering Challenges:

1) *High-Throughput Telemetry:* 10,000 IoT sensors transmit bursty fill-level events. 2) *Combinatorial Complexity:* CVRPTW optimization for 50 trucks is NP-hard. 3) *Fault Isolation:* Sensor network outages must not degrade driver manifests or citizen complaint intake.

PART A: ARCHITECTURAL JUSTIFICATION – MUNICIPAL SOLID WASTE ROUTE OPTIMIZER

> **Architecture Selection: "We chose Microservices Architecture for the Municipal Solid Waste Route Optimizer System."**

1. Architectural Choice

We selected the **Microservices Architecture** pattern for the Municipal Solid Waste Route Optimizer System. The system is decomposed into autonomous, independently deployable services: *IoT Ingestion Component*, *Route Optimizer Component*, *Fleet Dispatch Component*, *Citizen Complaint Component*, *Supervisor Portal Component*, and a central spatial *Waste Management DB Component*.

2. Two Specific Scenario-Related Reasons for Choice

- **Reason 1 (Decoupled Compute Scaling for Heuristic Route Optimization):** The platform's primary computational bottleneck is solving the NP-hard Capacitated Vehicle Routing Problem (CVRPTW) for 50 garbage trucks across 10,000 bins within a strict 45-second latency threshold (NFR-001). Under a Microservices Architecture, the *Route Optimizer Component* runs as an independent worker service on dedicated high-compute instances. It scales horizontally during peak morning scheduling bursts without competing for CPU, thread pools, or memory with high-frequency sensor ingestion or web dashboard traffic.
- **Reason 2 (Fault Isolation and Asynchronous Ingestion Resiliency):** Ultrasonic IoT sensors deployed in 10,000 public bins are vulnerable to wireless network dropouts, battery depletion, and burst transmission spikes. Isolating the *IoT Ingestion Component* behind an asynchronous message queue (RabbitMQ/Kafka) guarantees that telemetry bursts or gateway crashes are contained. Ingestion failures do not compromise the availability of the *Fleet Dispatch Component* (truck drivers continue navigating current manifests) or the *Citizen Complaint Component*.

3. Security Advantage

The Microservices Architecture implements a secure perimeter using an API Gateway with centralized Role-Based Access Control (RBAC). Public-facing endpoints utilized by citizens to lodge missed-pickup complaints are physically and logically segregated in a DMZ from privileged internal services (driver dispatch manifests and supervisor administrative controls). Furthermore, all IoT ultrasonic sensor communication is authenticated using per-sensor cryptographic tokens over mutual TLS (mTLS), strictly preventing sensor spoofing, rogue telemetry injection, or unauthorized route manipulation.

4. Performance Benefit

Microservices allow the *Route Optimizer Component* to leverage an in-memory spatial graph cache (Redis) containing pre-computed road network topologies, bin coordinates, and historical traffic patterns. By executing heuristic genetic algorithms and linear solvers entirely in memory rather than issuing blocking disk-bound SQL queries, the system completes route recalculation for 50 trucks across 10,000 bins in under 45 seconds, successfully fulfilling NFR-001 under peak load.

Architectural Styles Comparative Evaluation:

Architectural Style	Strengths for Waste Route Optimization	Weaknesses & Bottlenecks
Layered Architecture	Straightforward horizontal layering; easy to build and test initial prototype.	Monolithic business logic tier creates a critical bottleneck. Heavy route optimization CPU load blocks incoming sensor telemetry.
Client-Server Architecture	Centralized database management; straightforward synchronization of truck GPS.	Single point of failure at the central application server. Peak sensor bursts degrade server responsiveness for real-time truck tracking.
Microservices Architecture	Independent scaling of Route Optimization worker nodes; isolates sensor bursts; high fault tolerance; polyglot persistence.	Higher operational deployment complexity; requires message broker infrastructure and distributed tracing.

PART A: COMPONENT MODEL & UML DIAGRAM – MUNICIPAL SOLID WASTE ROUTE OPTIMIZER

Component Identification & Interface Specification Table (6 Components, 6 Interfaces):

Component Name	Stereotype & Responsibilities	Provided Interfaces (Ball)	Required Interfaces (Socket)
IoT Ingestion Component	«component» Ingests ultrasonic telemetry from 10,000 bins; validates sensor health; applies 80% threshold filter.	`IBinTelemetry` `[MQTT / HTTPS]`	`IRouteSolver` `IDataPersistence`
Route Optimizer Component	«component» Executes heuristic CVRPTW routing algorithms; computes optimal truck paths; re-evaluates dynamic pickups.	`IRouteSolver` `IRouteSchedule` `[gRPC / Internal API]`	`IDataPersistence` `[Spatial SQL]`
Fleet Dispatch Component	«component» Translates routes into driver manifests; pushes turn-by-turn guidance; tracks real-time truck GPS telematics.	`IDriverManifest` `[REST / WebSocket]`	`IRouteSchedule` `IDataPersistence`
Citizen Complaint Component	«component» Receives missed-pickup tickets; verifies bin status against manifest logs; injects emergency collection points.	`IComplaintService` `[HTTPS REST API]`	`IRouteSolver` `IDataPersistence`
Supervisor Portal Component	«component» Administrative web dashboard; displays bin fill heatmaps, live fleet tracking, route completion, and manual overrides.	`ISupervisorControl` `[HTTPS / WSS]`	`IRouteSolver` `IDataPersistence`
Waste Management DB Component	«database component» Central spatial and relational database; stores bin coordinates, truck specifications, historical routes, and audit logs.	`IDataPersistence` `[PostGIS / Spatial SQL]`	*None (Data Store)*

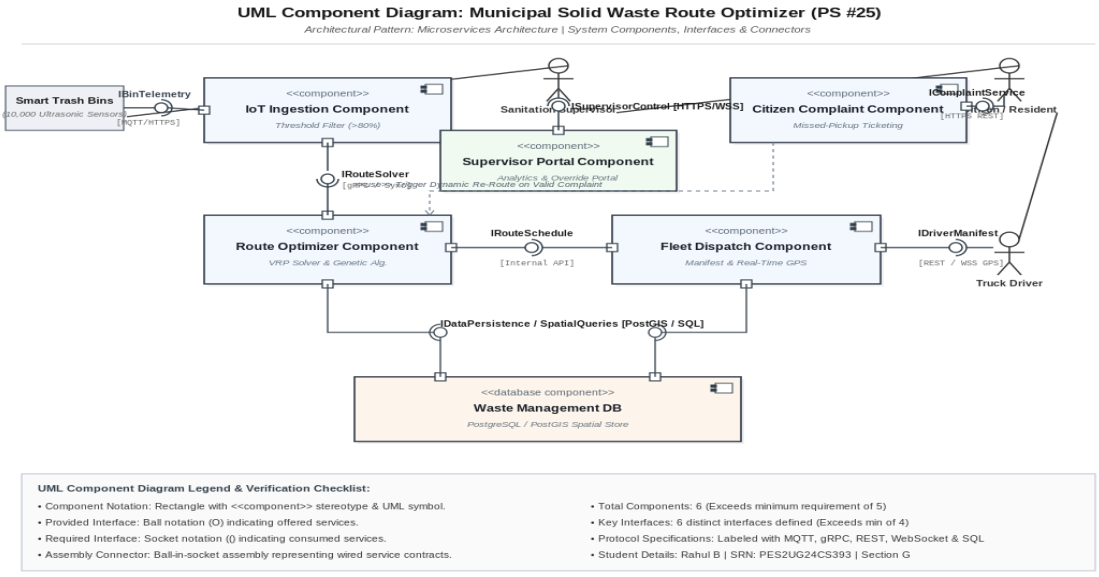


Figure 1: Comprehensive UML Component Diagram for Municipal Solid Waste Route Optimizer (PS #25)

PART B: IN-LAB HANDOUT SCENARIO – SELF-SERVICE COFFEE KIOSK SYSTEM

1. Scenario Review & Kiosk Specifications

Scenario Overview: Software architecture for an automated Self-Service Coffee Kiosk system in a high-traffic café environment. Customers interact directly with a physical touchscreen terminal to order beverages without human cashier intervention.

Core Functional Requirements & Constraints:

- *Coffee Selection:* Customers select from exactly 3 coffee types: **Espresso**, **Americano**, and **Latte**.
- *Drink Size:* Customers choose between 2 drink sizes: **Small** and **Large**.
- *Payment Processing:* Payment is accepted strictly via **Credit Card only** (no cash or mobile wallets).
- *Receipt Output:* System generates and physically prints receipts with complete order details.
- *Technical Constraints:* Touchscreen interaction; hardware receipt printer connection; local storage of menu data/pricing.

> **Architecture Selection: "We chose Layered Architecture for the Self-Service Coffee Kiosk System."**

2. Architectural Choice

We selected the **Layered Architecture** pattern for the Self-Service Coffee Kiosk System. The system is partitioned horizontally into Presentation (Touchscreen UI), Business Logic (Order Manager), Integration (Payment and Receipt Printer Services), and Data (Local SQLite DB) layers.

3. Two Specific Scenario-Related Reasons for Choice

- **Reason 1 (Simplicity and Standalone Embedded Execution):** The coffee kiosk operates as a self-contained embedded physical terminal on-site. All peripheral hardware devices (touchscreen, card reader, printer) are locally connected via USB/Serial interfaces. A Layered Architecture establishes a clean, maintainable separation of concerns across Presentation (UI), Business Logic (Order Manager), Integration (Hardware Drivers), and Data layers without the unneeded network overhead, deployment complexity, or network failure points of a distributed microservices design.
- **Reason 2 (Hardware Driver Abstraction and Modularity):** Kiosk hardware peripherals (such as the thermal receipt printer or EMV credit card reader) frequently require driver updates, model replacements, or hardware servicing. Layered architecture cleanly isolates peripheral communications into dedicated driver components (*Payment Service* and *Receipt Printer Service*), decoupling them from the core workflow engine (*Order Manager Component*). The physical printer can be replaced with an alternate hardware model by updating only the driver component, leaving order logic untouched.

4. Security Advantage

Strict layering enforces Payment Card Industry Data Security Standards (PCI-DSS) compliance. The *Payment Service Component* communicates directly with the physical EMV card reader hardware via Point-to-Point Encryption (P2PE). Cardholder credentials (card number, CVV) are encrypted at the hardware reader level and converted into transaction tokens. Plain card data is never exposed to the touchscreen UI, application logs, or local database, eliminating card skimming vulnerabilities.

5. Performance Benefit

All component interactions occur entirely in-process via compiled native interface calls and memory-mapped IPC rather than remote network calls. This guarantees ultra-responsive touchscreen feedback with touch-to-render latencies of under 30 milliseconds, preventing UI freezes during high-traffic café ordering rushes and enabling instant thermal receipt printing (<1 second) upon payment confirmation.

PART B: COMPONENT MODEL & UML DIAGRAM – SELF-SERVICE COFFEE KIOSK

Component Identification Table (5 Components including 2 Given in Handout):

Component Name	Layer & Responsibilities	Provided Interfaces (Ball)	Required Interfaces (Socket)
Touchscreen UI Component	**Presentation Layer** Renders customer menu; handles touch inputs for coffee selection and drink sizes; displays order summary and payment prompts.	`ITouchScreenInput` `[UI Events]`	`IOrderProcess` `IMenuData`
Order Manager Component *(Given in Handout)*	**Business Logic Layer** Controls ordering state machine; manages cart items; computes prices and taxes; triggers payment authorization and printing.	`IOrderProcess` `[In-Process API]`	`IPayment` `IPrintReceipt` `IMenuData`
Payment Service Component *(Given in Handout)*	**Integration Layer** Interfaces with physical credit card reader; transmits encrypted payment requests; verifies authorization tokens.	`IPayment` *(Given Interface)* `[Secure IPC API]`	`ICardHardware` `[USB / EMV HID]`
Receipt Printer Service	**Hardware Driver Layer** Formats receipt layout (timestamp, order ID, items, size, masked card info); generates ESC/POS commands for thermal printer.	`IPrintReceipt` `[Driver API]`	`IESCPOSControl` `[USB / Serial Port]`
Menu & Inventory DB	**Data Access Layer** Embedded SQLite data store; manages coffee catalog (Espresso, Americano, Latte), size multiplier prices, and ingredient inventory.	`IMenuData` `[SQLite / DAO]`	*None (Data Storage)*

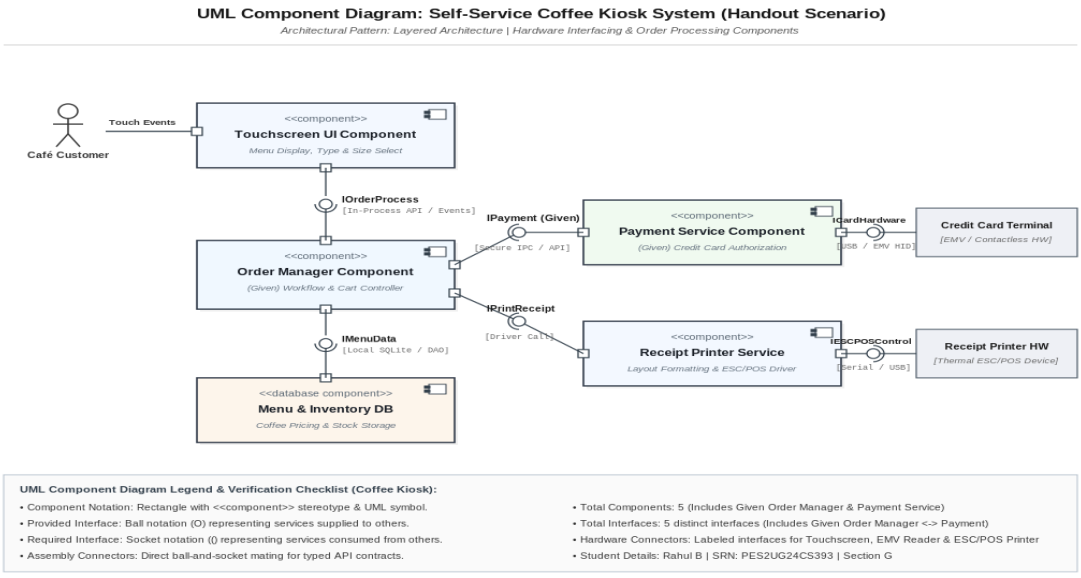


Figure 2: UML Component Diagram for Self-Service Coffee Kiosk System (Handout Scenario)

3. SUMMARY CHECKLIST & LEARNING OUTCOMES

Lab Deliverable Requirement	Municipal Solid Waste Route Optimizer (PS #25)	Self-Service Coffee Kiosk (Handout Scenario)
Architectural Style Selected	**Microservices Architecture**	**Layered Architecture**
Component Count	6 Components (Exceeds ≥ 5 requirement)	5 Components (Meets ≥ 5 requirement)
Component Inventory	IoT Ingestion, Route Optimizer, Fleet Dispatch, Citizen Complaint, Supervisor Portal, Waste Management DB	Touchscreen UI, Order Manager (Given), Payment Service (Given), Receipt Printer Service, Menu & Inventory DB
Interface Count	6 Interfaces (Exceeds ≥ 4 requirement)	5 Interfaces (Meets ≥ 4 requirement)
Mandatory Interface Covered	`IBinTelemetry`, `IRouteSolver`, `IDriverManifest`, `IDataPersistence`	`Order Manager ↔ Payment Service` (`IPayment`) explicitly modeled
UML Diagram Compliance	Full UML 2.x standard: ball-and-socket assembly connectors, ports, dependencies, stereotypes	Full UML 2.x standard: ball-and-socket assembly connectors, ports, dependencies, stereotypes
Written Justification Structure	Step 5 Template: Architecture Selection statement, 2 reasons, security advantage, performance benefit	Step 5 Template: Architecture Selection statement, 2 reasons, security advantage, performance benefit

Detailed Interface Verification:

1. **`IOrderProcess`:** Synchronous in-process native interface calls with event listeners connecting Touchscreen UI to Order Manager.
2. **`IPayment`:** Secure inter-process communication with encrypted transaction tokens connecting Order Manager to Payment Service.
3. **`IPrintReceipt`:** High-speed hardware driver API passing structured JSON receipt payloads connecting Order Manager to Receipt Printer Service.
4. **`IMenuData`:** Local SQLite data access object (DAO) query interface connecting Menu DB to Order Manager and Touchscreen UI.
5. **`ICardHardware` & `IESCPOSControl`:** Hardware device contracts driving physical EMV reader (USB HID) and thermal printer (ESC/POS serial).

Conclusion: In this laboratory session, we analyzed architectural trade-offs between monolithic Layered, distributed Microservices, and centralized Client-Server patterns. We established that large-scale municipal smart-city platforms with intensive heuristic computations (Problem Statement #25) require a Microservices Architecture for compute isolation, independent scaling, and fault tolerance. Conversely, standalone embedded physical appliances (Self-Service Coffee Kiosk) are best served by a Layered Architecture, providing clean hardware abstraction, zero network overhead, and instantaneous tactile response.